Rapport P1

Subject name 1 and Subject name 2

Projet MIDL 2 - Rapport de la 1ere Partie

Note sur l'usage des LLM dans le cadre du projet

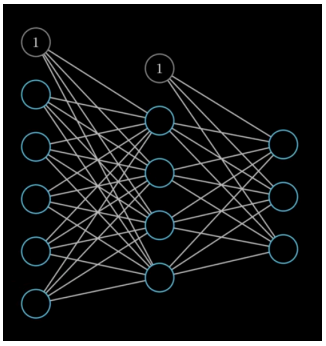
Plan (à modifier si besoin)

1. Les bases d'un réseau de neurones (Couche d'entrée, cachés, sortie, fonction d'activations)
2. [L'apprentissage d'un réseau] (Fonction de coût, descente de gradient, rétro propagation)
3. [Analyse des résultats] (impact des paramètres pour le temps d'apprentissage/précision)
4. [Références]

1. Les bases d'un réseau de neurones

1.1 - Les composantes principales

Un réseau de neurones est composé de plusieurs couches de neurones, qui se regroupent en 3 catégories : la couche d'entrée, où sont amenés les paramètres, une ou plusieurs couches intermédiaires, appelées couches cachées, qui servent au fonctionnement interne du réseau, et qui seront au cœur de l'apprentissage du modèle, et une couche de sortie, qui indiquera la réponse du système. D'un point de vue extérieur, cela ressemble à une fonction, mais l'expression de la fonction (par analogie, les couches cachées), se construisent pendant une phase d'apprentissage.



Source : Bot academy (Youtube)

Un neurone est constitué d'un nombre, ainsi que des connexions pondérées avec les neurones des autres couches. Dans l'exemple ci dessus, chaque neurone de la couche d'entrée a une connexion avec chaque neurone de la couche intermédiaire, qui ont eux même une connexion avec chaque neurone de la couche de sortie. On appelle ce schéma un réseau dense, ou entièrement connecté. On retrouve aussi des neurones biais, qui n'ont que des connexions avec les couches suivantes, et qui servent à activer ou non un neurone.

1.2 - Liens avec notre projet

L'objectif de cette première partie du projet est de créer un modèle simple permettant d'identifier un chiffre manuscrit écrit sur une grille de 28×28 pixels en nuances de gris. Nous utiliserons le jeu de données MNIST.

Un neurone sur la couche d'entrée aura pour valeur la nuance de gris d'un pixel (entre 0 et 255), qui sera réduit proportionnellement entre 0 et 1, afin de ne pas avoir d'explosions dans les calculs (nous y reviendrons). La couche d'entrée sera donc composée de $28^2 = 784$ neurones, les couches intermédiaires auront un nombre arbitraire de neurones (voir le § dédié), et la couche de sortie aura 10 neurones, qui exprimeront le chiffre prédit encodé sous le format *One-hot*, c'est à dire une matrice $M_{10,1}$ où chaque ligne i indique la probabilité que l'image d'entrée représente le chiffre $(i-1)$.

Exemple $P(x) = \begin{pmatrix} 0 \\ 0.9 \\ 0 \\ 0.1 \\ \dots \end{pmatrix}$ indique que l'image x à 90% de probabilité d'être un 1, et 10% de probabilité d'être un 3.

Remarque : les étiquettes seront aussi exprimées sous cette forme, avec une seule ligne valant 1, et les autres 0.

1.3 - Calcul des valeurs d'un neurone

Chaque neurone transmet sa valeur aux neurones de la couche supérieure, pondérée respectivement par le poids associé à chaque connexion.

Intéressons nous au calcul de la valeur d'un neurone, sur une couche cachée ou de sortie.

Dans un premier temps, on calcul une valeur intermédiaire, appelée signal de sortie.

On note $x_1, \dots x_n$ les valeurs des neurones la couche précédente, et $w_1, \dots, w_n$ les poids respectifs aux connexions, et b le biais associé au neurone correspondant. Alors la valeur d'un neurone s'exprime sous la forme

$$\left(\sum_{i=1}^{n} x_i w_i \right) + b$$

Mathématiquement, on représente les neurones d'une couche par une matrice horizontale. On note

$$X = (x_1 \quad \dots \quad x_n) \qquad \text{les valeurs des neurones recus,appelés signaux d'entrée}$$

$$W = \begin{pmatrix} w_{1,1} & \dots & \dots & w_{1,m} \\ & \dots & \dots & \\ w_{n,1} & \dots & \dots & w_{n,m} \end{pmatrix}$$

les poids des connexions, où $w_{i,j}$ est le poids de la connexion entre le neurone i de la couche précédente et le neurone j de la couche à calculer.

$$B = (b_1 \quad \dots \quad b_j) \qquad \text{le poids des connexions avec le neurone biais}$$

Ainsi, les valeurs intermédiares de la couche se calculent avec les operations matricielles $Z = XW + B$

Une fois ces calculs faits, les résultats passent par une *fonction d'activation*.

1.4 - Fonction d'activation

Une fonction d'activation est une fonction mathématique que l'on applique à un signal en sortie, et qui sert à activer ou non un neurone. Elle doivent répondre à plusieurs caractéristiques :

- **Non linéarité** : c'est ce qui permet à un modèle d'apprendre des motifs complexes (les modèles linéaires sont rares dans la nature, or les calculs précédents sont des opérations linéaires). De plus, sans fonction non linéaire, utiliser plus d'une couche intermédiaire serait inutile. Cela est donc nécessaire pour rendre le modèle plus précis.
- **Differentiable sur l'ensemble du domaine de définition** (nécessaire pour la descente de gradient)

Remarque : certaines fonctions d'activation usuelles modernes ont aussi une identité en 0, et ont une dérivée monotone.

Dans ce projet, nous avons pris en charge deux des fonctions les plus usuelles :

- Fonction sigmoïde :

  $$\sigma(x) = \frac{1}{1+e^{-x}} \quad \sigma'(x) = \sigma(x)(1-\sigma(x))$$

  Cette fonction est l'une des plus utilisées, et son domaine image étant $[0,1]$, on retrouve des résultats comparables à des probabilités.
- Fonction ReLu :

  $$\sigma(x) = \begin{cases} 0 & x < 0 \\ x & x \geqslant 0 \end{cases} \quad \sigma'(x) = \begin{cases} 0 & x < 0 \\ 1 & x \geqslant 0 \end{cases}$$

  Cette fonction, plus moderne, est plus simple à calculer, surtout pour sa dérivée.

Remarque : il existe une variante de cette fonction avec $\sigma(x) = max(\varepsilon x, x)$, afin d'avoir une dérivée positive pour tout x. Une dérivée qui peut être nulle empêche certaines couches cachées d'apprendre lors de la rétro-propagation. Nous ne l'utiliserons pas ici car les résultats avec ReLu classique sont satisfaisant.

- Fonction vectorielle softmax Soit $z = (z_1, \dots, z_k)$

  $$f(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{k} e^{z_j}}$$

  Etant une fonction vectorielle, nous n'avons à disposition que les dérivées partielles, qui seront utiles lors de la descente de gradient. Cette fonction renvoie, pour chaque composante du vecteur, un nombre compris entre 0 et 1, et la somme de tous les vecteurs vaut 1. Naturellement, c'est une fonction parfaite pour la couche de sortie

Démonstration

$$\sum_{i=1}^{k} softmax(z)_i = \sum_{i=1}^{k} \left(\frac{e^{z_i}}{\sum_{j=1}^{k} e^{z_j}} \right) = \frac{\sum_{i=1}^{k} e^{z_i}}{\sum_{j=1}^{k} e^{z_j}} = 1$$